SOFTWARE ARCHITECTURE DOCUMENT & EXECUTION ROADMAP

Ekosistem Smart School Terintegrasi

Dokumen Arsitektur Pengembangan Aplikasi, Development Plan (12 Bulan / 24 Sprint), dan Work Breakdown Task List

SERVICE A (75%)	SERVICE B (15%)	SERVICE C (10%)
Node.js + NestJS Core ERP, SSO RS256, LMS & Billing	Rust + Axum + SQLx CBT Engine, IoT MQTT & POS Wallet	Python + FastAPI OR-Tools Jadwal & EWS Analytics

SINGLE SOURCE OF TRUTH	APLIKASI KLIEN	PERANGKAT IOT
PostgreSQL 16+ (4 Skema) Redis Cluster & MinIO S3	React (Vite/TS) Web Flutter (Android & iOS)	EMQX MQTT + ESP32/RPi Gate RFID, POS, Kiosk, IP Bell

Versi: 1.0.0 **Tanggal:** 11 November 2026 **Basis:** Cetak Biru Ekosistem Smart School

Daftar Isi

1. BAB 1: Ringkasan Eksekutif, Pilar, & Prinsip Arsitektur
2. BAB 2: Arsitektur Sistem End-to-End & Topologi Layanan
3. BAB 3: Desain Mendalam Layanan Backend (Polyglot Services)
4. BAB 4: Desain Basis Data (Single Source of Truth) & Manajemen Penyimpanan
5. BAB 5: Alur Proses Bisnis Kritis Lintas Layanan (Hybrid Workflows)
6. BAB 6: Arsitektur Aplikasi Klien (Web, Mobile Flutter, & IoT)
7. BAB 7: Keamanan, Infrastruktur, Deployment, & Observability
8. BAB 8: Development Plan (Rencana Pengembangan 12 Bulan / 24 Sprint)
9. BAB 9: Task List & Work Breakdown Structure (WBS)

BAB 1: Ringkasan Eksekutif, Pilar, & Prinsip Arsitektur

1.1 Tujuan dan Empat Pilar Ekosistem

Dokumen ini menjabarkan spesifikasi arsitektur pengembangan aplikasi, rencana eksekusi 12 bulan, serta daftar tugas (*task list*) implementasi **Ekosistem Smart School Terintegrasi**. Sistem dirancang untuk menopang empat pilar utama operasional sekolah modern:

1. **Integrasi Akademik**: Penyatuan data mulai dari penerimaan siswa (E-PPDB), Buku Induk Digital, SIAKAD, LMS, CBT, hingga E-Rapor ber-Tanda Tangan Elektronik (TTE) tanpa *data silos*.
2. **Kemudahan Layanan Siswa & Orang Tua**: Akses terpadu melalui Mobile Super-App untuk absensi *real-time*, pemantauan nilai, pengajuan izin, pembayaran SPP, hingga dompet *cashless* kantin dengan batas jajan harian.
3. **Efisiensi Kerja Guru & Tendik**: Penyusunan jadwal pelajaran otomatis bebas bentrok, presensi berbasis GPS + swafoto, pengisian jurnal mengajar di bawah 1 menit, serta *auto-grading* ujian CBT.
4. **Transparansi Manajemen Sekolah**: Dashboard Eksekutif bagi Kepala Sekolah dan Yayasan yang dilengkapi *Early Warning System* (EWS) untuk deteksi dini risiko akademik dan kedisiplinan siswa, serta transparansi tata kelola keuangan (SPP & dana BOS).

1.2 Tiga Prinsip Mutlak Arsitektur

Seluruh keputusan teknis dan desain modul wajib mematuhi tiga prinsip mutlak berikut:

Prinsip	Implementasi Teknis	Dampak Operasional
1. Single Source of Truth (SSOT)	Satu klaster **PostgreSQL 16+** sebagai pusat data tunggal yang dipecah ke dalam 4 skema logis (`iam_master`, `academic_lms`, `high_speed_ops`, `finance_analytics`) dengan aturan *Write Ownership* yang ketat.	Perubahan di satu titik (mis. kelulusan E-PPDB) otomatis memperbarui Buku Induk, akun LMS, dan dompet kantin tanpa input ulang dan menjaga integritas *Foreign Key*.
2. Single Sign-On (SSO) & RBAC	Satu kredensial akun untuk seluruh warga sekolah (Kepala Sekolah, Guru, Staf TU, Siswa, Orang Tua, Merchant) menggunakan token **JWT Asimetris (RS256)**.	`private.pem` hanya dipegang Service A (Node.js); Service B (Rust) dan Service C (Python) memverifikasi token secara lokal memakai `public.pem` (*read-only*) tanpa beban kueri DB.

Prinsip	Implementasi Teknis	Dampak Operasional
3. Polyglot Service-Oriented Architecture	Pemisahan beban kerja backend ke tiga runtime sesuai karakteristik komputasi: **Node.js (NestJS)** ~75%, **Rust (Axum)** ~15%, dan **Python (FastAPI)** ~10%.	Mencegah kegagalan server saat *burst traffic* (ujian CBT serentak, tap RFID pagi, jam istirahat kantin) sekaligus menjaga kecepatan pengembangan fitur ERP.

1.3 Target Non-Functional Requirements (NFR) & SLA

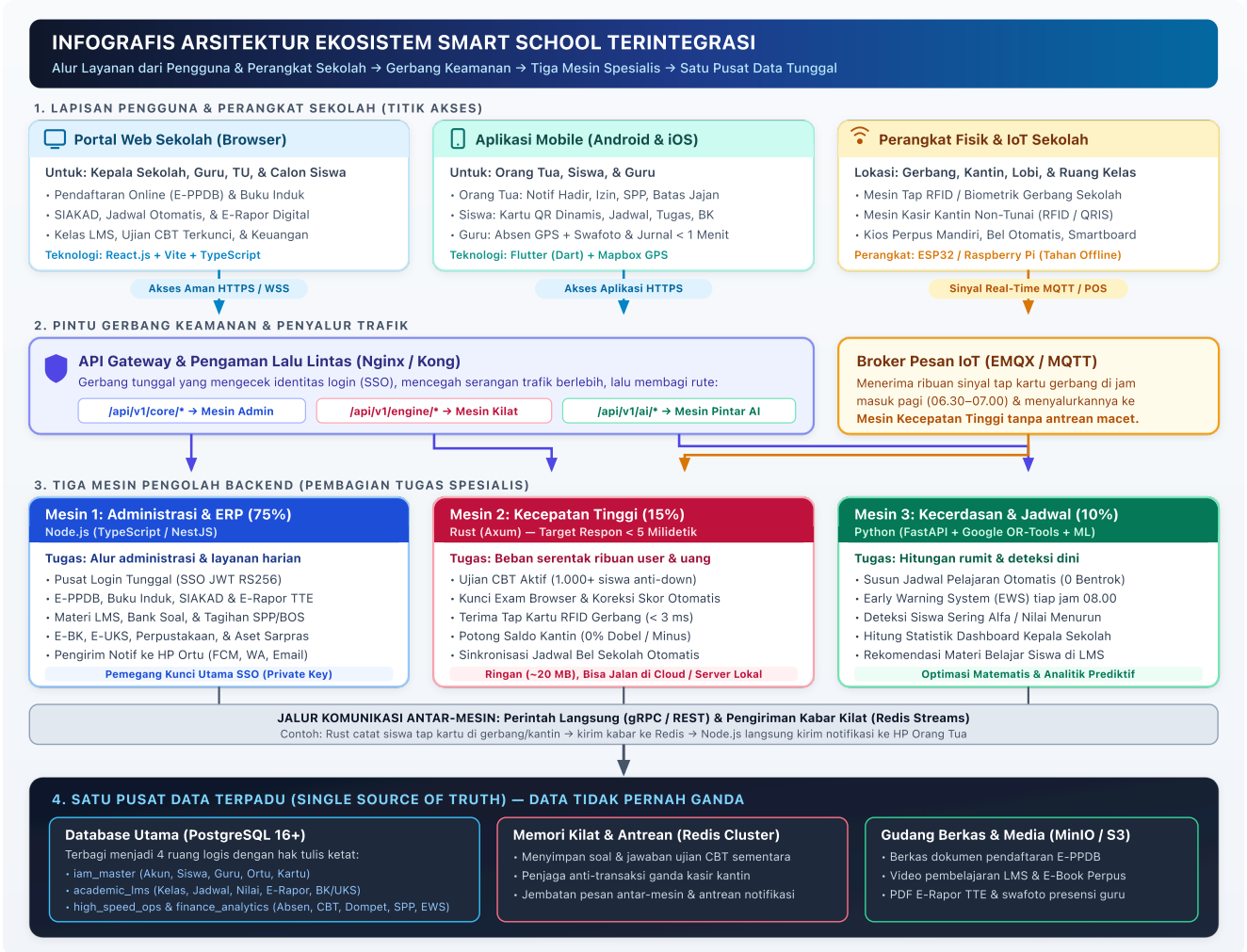
Parameter	Target SLA / Batas Kinerja	Komponen Penanggung Jawab
Latensi Ingesti IoT RFID	`< 3 ms` per event validasi tap kartu (MQTT → DB/Cache → Event)	Service B (Rust / Axum) + EMQX
Latensi Transaksi POS Kantin	`< 5 ms` per transaksi potong saldo atomik (`FOR UPDATE`)	Service B (Rust / Axum) + PostgreSQL
Kapasitas Ujian CBT Serentak	`≥ 1.000` siswa aktif bersamaan per node dengan utilisasi CPU & RAM `< 20%`	Service B (Rust) + Redis Cluster
Latensi API ERP Standar	`P95 < 200 ms` untuk endpoint CRUD SIAKAD, PPDB, Keuangan	Service A (Node.js / NestJS)
Keandalan Finansial POS	`0%` saldo minus, `0%` *double-deduct* (*race condition* / *double-tap*)	Rust + PostgreSQL `BIGINT` + `idempotency_key`
Resiliensi Offline Perangkat	Terminal ESP32 menyimpan antrean tap di *flash* lokal & *batch sync* otomatis	Firmware ESP32 + Rust Batch Endpoint
Fleksibilitas Deployment Edge	Binari kompilasi Rust `~20 MB` siap deploy di Cloud VPS maupun Mini-Server On-Premise	Service B (Rust Static/Minimal Binary)

BAB 2: Arsitektur Sistem End-to-End & Topologi Layanan

2.1 Infografis Arsitektur Ekosistem (Gambaran Umum & Teknis)

Agar mudah dipahami baik oleh manajemen sekolah (non-teknis) maupun tim pengembang, sistem ini bekerja layaknya **gedung layanan terpadu dengan satu pintu masuk dan satu brankas data**:

1. **Lapisan Pengguna & Perangkat:** Kepala Sekolah, Guru, Staf TU, Siswa, dan Orang Tua mengakses sistem melalui **Web** atau **Aplikasi HP**, sementara aktivitas fisik di sekolah (tap kartu gerbang, kasir kantin, kios perpus, bel sekolah) ditangani oleh **Perangkat IoT**.
2. **Pintu Gerbang Keamanan:** Seluruh permintaan masuk diperiksa terlebih dahulu oleh **API Gateway** (untuk Web/Mobile) dan **Broker EMQX** (untuk ribuan sinyal IoT) agar aman dan langsung diarahkan ke mesin yang tepat.
3. **Tiga Mesin Spesialis:** Beban kerja dibagi rata ke tiga mesin sesuai keahliannya—**Mesin Administrasi** (*Node.js*, 75% fitur harian), **Mesin Kecepatan Tinggi** (*Rust*, 15% fitur padat seperti ujian CBT serentak, tap gerbang, dan kasir kantin `< 5 ms`), serta **Mesin Kecerdasan & Jadwal** (*Python*, 10% fitur penjadwalan otomatis & deteksi dini siswa).
4. **Satu Pusat Data Terpadu (SSOT):** Ketiga mesin menyimpan dan membaca data dari satu pusat penyimpanan tunggal (**PostgreSQL**, **Redis**, dan **MinIO**) sehingga tidak pernah terjadi data ganda.



Infografis Arsitektur Ekosistem Smart School Terintegrasi

2.2 Spesifikasi API Gateway & Routing

API Gateway (Kong atau Nginx) menjadi satu-satunya titik masuk HTTP/HTTPS/WebSocket dari jaringan publik:

Prefix Rute	Layanan Tujuan	Kebijakan Rate Limit & Timeout	Catatan Khusus
`/api/v1/core/auth/*`	Service A (Node.js)	Ketat (mis. `20 req/menit/IP` untuk login/OTP)	Proteksi *brute-force* & *credential stuffing*
`/api/v1/core/webhooks/*`	Service A (Node.js)	Whitelist IP Payment Gateway + HMAC Signature	Bypass rate limit user, validasi tanda tangan wajib
`/api/v1/core/*`	Service A (Node.js)	`120 req/menit/user`, timeout `30s` (ekspor/TTE `60s`)	Upload file besar diarahkan via S3/MinIO *Presigned URL*
`/api/v1/engine/cbt/*`	Service B (Rust)	Burst tinggi (`100 req/detik/kelas`), WebSocket upgrade	Koneksi persisten untuk sinkronisasi jawaban & *heartbeat*
`/api/v1/engine/pos/*`	Service B (Rust)	`60 req/detik/terminal`, timeout `5s`	Wajib menyertakan header `Idempotency-Key`
`/api/v1/engine/iot/*`	Service B (Rust)	Autentikasi mTLS / Device API Key	Jalur *batch sync* untuk perangkat ESP32 saat pulih offline
`/api/v1/ai/*`	Service C (Python)	`30 req/menit/user`, timeout `10s` (async job)	Tugas berat dijalankan asinkron dan dilaporkan via Redis

2.3 Topologi Broker IoT (EMQX / MQTT)

EMQX melayani komunikasi mesin-ke-mesin berlatensi rendah di lingkungan sekolah:

- **Protokol**: MQTT v5.0 over TLS (port `8883`) atau TCP internal VLAN IoT (`1883`).
- **Struktur Topik Standar**:
- `school/{school_id}/gate/{device_id}/attendance` (QoS 1 — *at least once* untuk menjamin tap kartu tidak hilang).
- `school/{school_id}/gate/{device_id}/ack` (QoS 0 — balasan status LED/buzzer terminal: Hijau/Merah/Terlambat).
- `school/{school_id}/bell/{device_id}/schedule` (QoS 1, *Retained* — jadwal bunyi bel terbaru).
- `school/{school_id}/device/{device_id}/heartbeat` (QoS 0 — status kesehatan & *firmware version* setiap 60 detik).
- **Integrasi ke Service B**: Rust menjalankan *shared subscription* (`$share/rust_workers/school/+/gate/+/attendance`) sehingga beberapa replika pod Rust dapat membagi beban ratusan pesan per detik secara paralel tanpa duplikasi pemrosesan.

BAB 3: Desain Mendalam Layanan Backend (Polyglot Services)

3.1 Service A — Core ERP & Academic Service (`Node.js / TypeScript / NestJS`)

Menangani **~75% cakupan fungsional** aplikasi yang berfokus pada logika bisnis kompleks, alur persetujuan, manajemen dokumen, dan integrasi pihak ketiga.

- **Tanggung Jawab Utama**:

1. **Pusat SSO & IAM**: Registrasi/aktivasi akun warga sekolah, manajemen RBAC (Role, Permission, Role Assignment), penerbitan JWT RS256 (Access Token berumur pendek 15 menit + Refresh Token rotasi di Redis), manajemen *pin* transaksi kantin, serta manajemen siklus kartu identitas (`identity_cards`).
2. **E-PPDB & Buku Induk**: Portal pendaftaran, unggah berkas ke MinIO via *presigned URL*, verifikasi admin, pembuatan tagihan biaya pendaftaran (VA/QRIS), orkestrasi hasil *Placement Test* dari Rust, pengumuman kelulusan, dan **One-Click Migration** (memindahkan data calon siswa ke tabel `students` , membuat akun LMS, dan menginisialisasi `wallets`).
3. **SIAKAD & E-Rapor**: Manajemen tahun ajaran, rombongan belajar (rombel), mata pelajaran, sinkronisasi Dapodik/EMIS, rekap nilai dari LMS/CBT, penerbitan PDF E-Rapor, integrasi Tanda Tangan Elektronik (TTE/BSrE atau penyedia tersertifikasi), dan portofolio siswa.
4. **LMS (Course Management)**: Pembuatan materi, *streaming* video via S3/MinIO, tugas/PR, forum diskusi, dan manajemen **Bank Soal** (menyimpan soal ke tabel `cbt_exams` dan `cbt_questions` sebelum dieksekusi oleh Rust).
5. **E-Finance & Billing**: Pembuatan tagihan SPP/uang gedung (`invoices`), penanganan webhook Payment Gateway (VA/QRIS), jurnal akuntansi sekolah, penggajian (*payroll*) guru & tendik, serta pelaporan dana BOS.
6. **Layanan Kesiswaan, Perpustakaan & Sarpras**: E-BK (poin disiplin/prestasi, catatan konseling terenkripsi), E-UKS (rekam kunjungan medis, alergi/golongan darah), Alumni Tracer Study, E-Library (katalog OPAC, sirkulasi pinjam/kembali, e-book), dan Asset Management (QR inventaris, booking ruangan, tiket kerusakan).
7. **Notification Dispatcher & Event Consumer**: Mengonsumsi Redis Streams (`AttendanceRecorded` , `WalletDeducted` , `ScheduleGenerated` , `EwsAlertRaised`) dan mendistribusikan notifikasi melalui **Firebase Cloud Messaging (FCM)**, **WhatsApp Gateway**, dan **Email** menggunakan antrean **BullMQ** agar pengiriman bervolume tinggi tidak memblokir API utama.

- **Struktur Direktori Modular yang Disarankan (`services/core-erp`)**:

```
src/
├── common/             # Guards (RS256 JWT, RBAC), Interceptors, MinIO, Redis clients
├── modules/
│   ├── iam/            # Auth, Users, Roles, Identity Cards
│   ├── ppdb/           # Pendaftaran, Verifikasi, One-Click Migration
│   ├── siakad/         # Buku Induk, Rombel, Jadwal, Nilai, E-Rapor TTE, Dapodik
│   ├── lms/            # Materi, Tugas, Forum, Bank Soal CBT
│   ├── finance/        # Invoices, Payment Gateway Webhook, BOS, Payroll
│   ├── student-care/   # E-BK, E-UKS, Alumni Tracer Study
│   ├── library-asset/  # OPAC, Sirkulasi, Aset QR, Sarpras Ticketing
│   └── notifications/  # Redis Stream Consumers + BullMQ Dispatchers (FCM/WA/Email)
└── main.ts
```

3.2 Service B — High-Concurrency & Transaction Engine (`Rust / Axum / SQLx / Tokio`)

Menangani **~15% modul paling kritis** yang membutuhkan latensi `< 5 ms` , ketahanan terhadap *burst traffic*, dan jaminan keamanan memori (*memory safety*) tanpa *garbage collector pause*.

- **Tanggung Jawab Utama**:

1. **Engine Ujian CBT Aktif**:

- *Pre-warming* (H-15 menit sebelum jadwal ujian): Mengambil paket soal dari PostgreSQL (`cbt_exams` , `cbt_questions`), mengenkripsi *payload* soal, dan menyimpannya ke Redis Cluster.
- *Distribusi & Pengacakan*: Saat 1.000+ siswa klik "Mulai Ujian", Axum mengacak urutan soal/opsi secara deterministik per `student_id` dan mengirimkan paket soal.
- *Real-time Answer Sync & Exam Lock*: Menerima jawaban per butir melalui WebSocket / Fast HTTP, menyimpan ke Redis Hash (`cbt:session:{exam_id}:{student_id}`), memvalidasi `exam_browser_key` & *heartbeat* anti-kecurangan (mencatat `cheat_violation_count`), serta melakukan *batch flush* periodik ke PostgreSQL (`cbt_student_submissions`).
- *Auto-Grading Mikrodetik*: Begitu siswa menekan "Selesai" (atau waktu habis), Rust langsung menghitung skor objektif di memori, menyimpan `auto_graded_score` ke PostgreSQL, dan mempublikasikan event `CbtExamSubmitted` .

1. **Ingesti Absensi IoT (RFID / Biometrik)**:

- Berlangganan topik MQTT EMQX secara asinkron dengan Tokio.
- Memvalidasi `rfid_uid` terhadap cache Redis (fallback cepat ke `iam_master.identity_cards` via SQLx prepared statement).
- Mengevaluasi aturan jam masuk sekolah (`PRESENT` vs `LATE`), mencatat baris ke `high_speed_ops.attendances` , mengirim ACK balik ke perangkat MQTT, dan mempublikasikan `AttendanceRecorded` ke Redis Streams dalam `< 3 ms` .
- Menyediakan endpoint HTTP *batch ingestion* untuk menerima log tertunda dari ESP32 setelah koneksi internet/Wi-Fi pulih.

1. **Core Wallet & POS Kantin Cashless**:

- Menyediakan endpoint `/api/v1/engine/pos/pay` yang menerapkan transaksi PostgreSQL dengan *Pessimistic Row-Level Locking* (`SELECT ... FOR UPDATE` pada baris `wallets`).
- Memeriksa **Idempotency Key** terlebih dahulu (di Redis + constraint `UNIQUE` PostgreSQL `wallet_transactions.idempotency_key`) untuk menggagalkan tap ganda di detik yang sama.
- Memvalidasi saldo (`balance >= amount`) dan akumulasi jajan harian terhadap `daily_spending_limit` (dalam satuan `BIGINT` rupiah penuh tanpa koma/floating point).
- Memotong saldo siswa, menambah saldo merchant kantin secara atomik dalam satu transaksi ACID, mempublikasikan event `WalletDeducted` (beserta `items_summary_jsonb`) ke Redis Streams, dan membalas kasir POS dalam `< 5 ms` .

1. **Sinkronisasi Automated IP Bell**: Menyajikan jadwal bunyi bel ringan hasil sinkronisasi dari `academic_lms.schedules` dan mendorong jadwal ke perangkat bel jaringan melalui MQTT/HTTP lokal.

- **Struktur Direktori yang Disarankan (`services/high-speed-engine`)**:

```
src/
├── config/         # Env, Public Key RS256 loader, DB & Redis pools
├── auth/           # Middleware verifikasi JWT RS256 lokal (tanpa DB hit)
├── cbt/            # Pre-warmer, WS/HTTP session handler, exam-lock validator, auto-grader
├── iot/            # EMQX MQTT worker, batch HTTP sync, IP Bell scheduler
├── wallet/         # Atomic POS checkout, idempotency guard, daily limit checker
└── main.rs         # Axum router & graceful shutdown
```

3.3 Service C — Intelligence, Scheduling & Analytics Service (`Python / FastAPI`)

Menangani **~10% modul** yang menuntut komputasi matematis berat (*CPU-bound*), pemrograman kendala (*constraint programming*), dan analitik prediktif berbasis data historis.

- **Tanggung Jawab Utama**:

1. **Auto-Scheduling Engine (Google OR-Tools CP-SAT Solver)**:

- Menerima parameter dari Node.js (periode akademik, daftar rombel, beban jam mengajar guru, mata pelajaran, ketersediaan laboratorium/ruang khusus, preferensi waktu/hari kosong guru).
- Memodelkan *Hard Constraints* (guru tidak boleh mengajar 2 kelas sekaligus, kelas tidak boleh punya 2 pelajaran sekaligus, ruang lab tidak boleh bentrok, batas jam maksimal harian) dan *Soft Constraints* (sebaran mapel eksakta di jam pagi, menghindari jeda kosong panjang bagi guru).
- Menjalankan solver secara asinkron (Worker/ThreadPool agar event loop FastAPI tetap responsif), menulis jadwal final ke `academic_lms.schedules`, dan mengirim event `ScheduleGenerated` ke Redis Streams.

1. **Early Warning System (EWS) Engine**:

- Job terjadwal harian pukul **08.00 WIB** (serta pemicu manual oleh Kepala Sekolah/Guru BK).
- Menggunakan **Pandas** & **Scikit-learn** untuk memindai histori `high_speed_ops.attendances` (pola alfa beruntun, tren keterlambatan) dan `academic_lms.grades` (penurunan nilai signifikan lintas mata pelajaran).
- Menghitung `attendance_risk_score` (0–100), `academic_drop_score` (0–100), mengklasifikasikan `risk_level` (`LOW`, `MEDIUM`, `CRITICAL`), menghasilkan `recommended_action`, dan menulis ke `finance_analytics.ews_student_risk_scores`.

1. **Agregasi Analitik Eksekutif & Rekomendasi LMS**: Menyediakan endpoint agregasi statistik lintas semester/angkatan untuk Dashboard Yayasan & Kepala Sekolah serta rekomendasi materi remedial berbasis capaian nilai siswa.

3.4 Kontrak Komunikasi Antar-Layanan

A. Komunikasi Sinkron (gRPC / Internal REST over Private Network)

Pemanggil	Penerima	Endpoint / RPC	Tujuan
Service A (Node.js)	Service C (Python)	`POST /internal/ai/schedules/generate`	Memicu kalkulasi jadwal OR-Tools (mengembalikan `job_id` status `PROCESSING`)
Service A (Node.js)	Service C (Python)	`GET /internal/ai/analytics/executive-summary`	Mengambil hasil agregasi tren akademik & EWS untuk Dashboard
Service A (Node.js)	Service B (Rust)	`POST /internal/engine/cbt/{exam_id}/prewarm`	Memicu pre-warming soal CBT ke Redis secara manual/terjadwal
Service A (Node.js)	Service B (Rust)	`POST /internal/engine/wallets/credit`	Menambah saldo dompet saat webhook top-up QRIS/VA terkonfirmasi secara atomik

B. Komunikasi Asinkron (Redis Streams Event Bus)

Seluruh event menggunakan amplop JSON seragam: `{ "event_id": "uuid", "event_type": "...", "occurred_at": "ISO8601", "source": "rust-engine|python-ai|node-erp", "payload": { ... } }`.

Stream / Event Type	Produser	Konsumen Utama	Aksi Lanjutan
`AttendanceRecorded`	Service B (Rust)	Service A (Node.js)	Kirim Push Notification FCM/WhatsApp ke orang tua siswa terkait

Stream / Event Type	Produser	Konsumen Utama	Aksi Lanjutan
`WalletDeducted`	Service B (Rust)	Service A (Node.js)	Kirim rincian jajanan (`items_summary_jsonb`) & sisa saldo ke Parent App
`CbtExamSubmitted`	Service B (Rust)	Service A (Node.js)	Sinkronkan nilai CBT ke rekap penilaian SIAKAD & log aktivitas LMS
`ScheduleGenerated`	Service C (Python)	Service A (Node.js) & Service B (Rust)	Node.js notif ke Guru/Siswa; Rust sinkron jadwal bunyi Automated IP Bell
`EwsAlertRaised`	Service C (Python)	Service A (Node.js)	Tampilkan notifikasi prioritas ke Dashboard Kepala Sekolah & Guru BK

BAB 4: Desain Basis Data (Single Source of Truth) & Manajemen Penyimpanan

4.1 Matriks Kepemilikan Skema PostgreSQL 16+

Untuk menjaga integritas relasional (*Foreign Key* antar domain) tanpa menciptakan *tight coupling* pada kode penulisan, satu klaster PostgreSQL dibagi ke dalam **4 Skema Logis** dengan aturan kepemilikan tulis (*Write Ownership*) yang ketat:

Skema Logis	Hak Tulis (Write Owner)	Hak Baca (Read Access)	Domain Data
** `iam_master` **	**Service A (Node.js)**	Service B (Rust), Service C (Python)	Pengguna, RBAC, Kartu RFID/QR, Data Induk Siswa, Guru/Tendik, Orang Tua
** `academic_lms` **	**Service A (Node.js)** & **Service C (Python)** *(khusus tabel `schedules`)*	Service B (Rust)	Tahun ajaran, rombel, mapel, jadwal, jurnal mengajar, nilai, E-Rapor, E-BK, E-UKS, Perpus, Sarpras
** `high_speed_ops` **	**Service B (Rust)** *(kecuali `cbt_exams` / `cbt_questions` oleh Node.js)*	Service A (Node.js), Service C (Python)	Presensi real-time, paket/jawaban CBT, dompet elektronik & buku besar transaksi
** `finance_analytics` **	**Service A (Node.js)** *(tagihan `invoices`) & ***Service C (Python)*** (skor `ews_student_risk_scores`)*	Service A, B, C sesuai kebutuhan	Tagihan SPP/pembayaran sekolah & skor risiko EWS

4.2 Spesifikasi Tabel Inti per Skema

1. Skema `iam_master`

- `users`: `id` (UUID, PK), `username` (Unique), `email` (Unique), `phone`, `password_hash`, `pin_hash` (untuk PIN kantin/otorisasi), `is_active`, `last_login_at`.
- `roles`, `permissions`, `user_roles`: Konfigurasi RBAC lintas portal Web dan aplikasi Mobile.
- `identity_cards`: `id` (UUID, PK), `user_id` (FK → `users.id`), `rfid_uid` (Unique Index — di-cache & dibaca cepat oleh Rust), `qr_token`, `card_status` (`ACTIVE`, `LOST`, `BLOCKED`).
- `students`: `id` (UUID, PK), `user_id` (FK), `nis` (Unique), `nisn` (Unique), `nik`, `full_name`, `gender`, `birth_date`, `blood_type`, `allergy_notes`, `status` (`CANDIDATE`, `ACTIVE`, `GRADUATED`, `TRANSFERRED`), `dapodik_sync_id`.
- `teachers_and_staff`: `id` (UUID, PK), `user_id` (FK), `nip_nuptk`, `full_name`, `employment_status`, `position`.
- `parents` & `student_parent_relations`: Relasi keluarga (`FATHER`, `MOTHER`, `GUARDIAN`, `is_primary_contact`) untuk menentukan penerima notifikasi dan pengelola batas jajan siswa.

2. Skema `academic_lms`

- `academic_periods`, `classrooms`, `classroom_members`, `subjects`: Master struktur tahun ajaran, kelas, penempatan siswa, dan kurikulum.
- `schedules` *(Ditulis oleh Python OR-Tools & disesuaikan Node.js)*: `id` (UUID, PK), `academic_period_id` (FK), `classroom_id` (FK), `subject_id` (FK), `teacher_id` (FK), `room_code`, `day_of_week`, `start_time`, `end_time`.
- `teaching_journals`: Jurnal harian mengajar guru dari Teacher App (`teacher_id`, `schedule_id`, `topic`, `notes`, `logged_at`).
- `grades` & `e_rapor_documents`: Nilai akhir per kompetensi, deskripsi capaian, serta `tte_signed_pdf_url` (tautan PDF E-Rapor di MinIO yang sudah ber-TTE).
- `counseling_records` (E-BK — kolom catatan konseling dienkripsi di tingkat aplikasi), `uks_visits` (E-UKS), `library_catalogs`, `library_loans`, `assets`, `maintenance_tickets`.

3. Skema `high_speed_ops`

- `attendances`: `id` (BIGSERIAL, PK), `user_id` (FK → `iam_master.users.id`), `date` (DATE), `check_in_time`, `check_out_time`, `status` (`PRESENT`, `LATE`, `SICK`, `PERMIT`, `ALPHA`), `verification_method` (`RFID_GATE`, `BIOMETRIC`, `GPS_MOBILE`), `iot_device_id`. *(Indeks komposit pada `(date, user_id)` untuk kueri cepat pagi hari & job EWS).*
- `cbt_exams` & `cbt_questions` *(Ditulis oleh Node.js saat guru menyusun ujian)*: `id` (UUID, PK), `subject_id` (FK), `title`, `start_at`, `duration_minutes`, `is_random`, `exam_browser_key`.
- `cbt_student_submissions` *(Ditulis intensif oleh Rust)*: `id` (UUID, PK), `exam_id` (FK), `student_id` (FK), `answers_jsonb` (JSONB), `auto_graded_score` (NUMERIC), `cheat_violation_count` (INT), `submitted_at`. *(Constraint `UNIQUE(exam_id, student_id)`).*
- `wallets` *(Diakses atomik oleh Rust)*: `id` (UUID, PK), `user_id` (FK, Unique), `balance` (**BIGINT** — dalam satuan rupiah penuh tanpa desimal untuk presisi mutlak), `daily_spending_limit` (**BIGINT**), `version` (INT — mendukung *optimistic/pessimistic locking*).
- `wallet_transactions` *(Ditulis oleh Rust)*: `id` (UUID, PK), `wallet_id` (FK), `transaction_type` (`TOPUP`, `POS_PURCHASE`, `LIBRARY_FINE`, `MERCHANT_SETTLEMENT`), `amount` (**BIGINT**), `merchant_user_id` (FK), `items_summary_jsonb` (JSONB), `idempotency_key` (Unique Index), `created_at`.

4. Skema `finance_analytics`

- `invoices` *(Dikelola Node.js)*: `id` (UUID, PK), `invoice_number` (Unique), `student_id` (FK), `billing_type`, `amount` (**BIGINT**), `due_date`, `status` (`UNPAID`, `PAID`, `EXPIRED`), `va_number`, `qris_string`, `paid_at`.
- `ews_student_risk_scores` *(Dihasilkan Python Analytics Engine)*: `id` (UUID, PK), `student_id` (FK), `calculated_date`, `attendance_risk_score` (`0–100`), `academic_drop_score` (`0–100`), `risk_level` (`LOW`, `MEDIUM`, `CRITICAL`), `recommended_action`, `is_resolved_by_bk` (BOOLEAN).

4.3 Strategi Migrasi Database Terpusat

Karena ada tiga bahasa pemrograman yang mengakses satu database PostgreSQL yang sama:

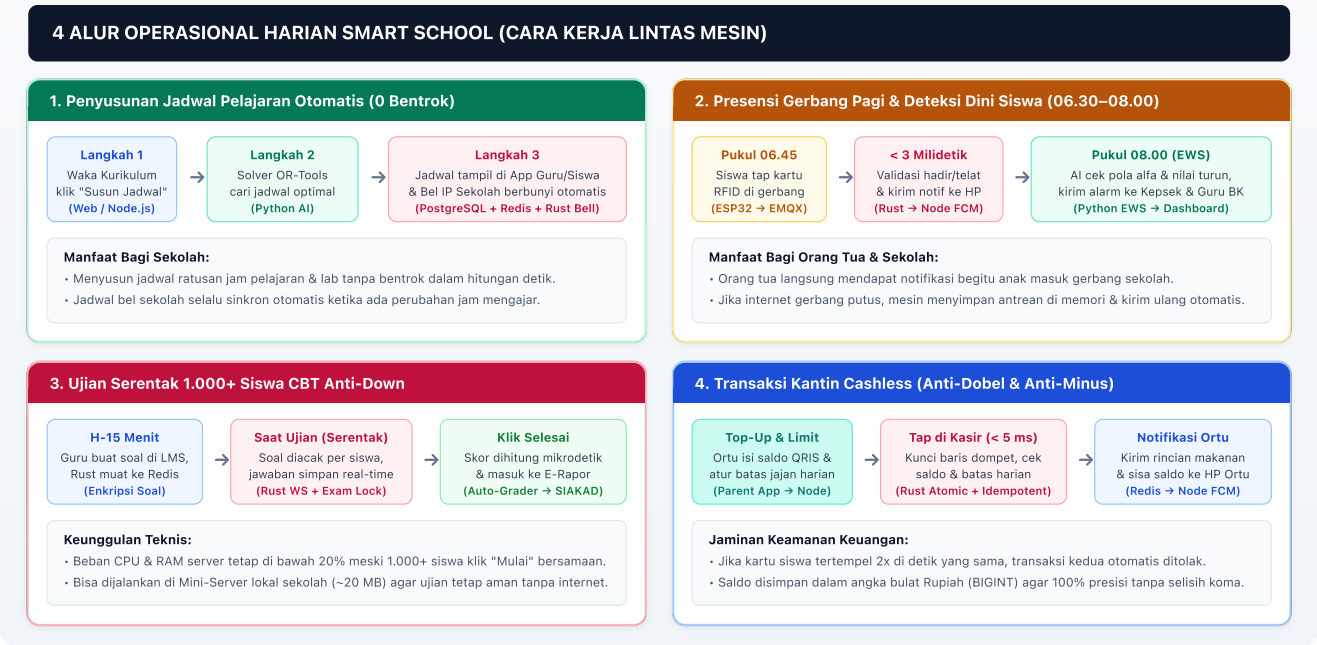
- **Single Migration Authority**: Seluruh perubahan skema DDL (`CREATE TABLE`, `ALTER TABLE`, `CREATE INDEX`) dikelola dalam satu folder migrasi SQL terpusat (`infra/migrations/`) menggunakan tool berbasis SQL murni seperti **Atlas** atau **golang-migrate / dbmate**).
- Layanan Node.js, Rust (`sqlx::query!`), dan Python (`SQLAlchemy/psycopg`) dilarang membuat skrip migrasi terpisah yang saling menimpa. Pipeline CI/CD menjalankan migrasi SQL terpusat di *staging/production* sebelum *rollout* kontainer layanan, dan memvalidasi kompilasi SQLx Rust terhadap skema terbaru.

4.4 Manajemen Redis Cluster & Object Storage (MinIO / S3)

Infrastruktur	Struktur Keyspace / Bucket	Kebijakan Retensi & Keamanan
Redis Cluster	`iam:rfid:{rfid_uid}` (Cache kartu aktif)	TTL 24 jam, di-invalidasi Node.js saat kartu diblokir/diganti
Redis Cluster	`cbt:exam:{exam_id}:payload` (Soal terenkripsi)	Diisi oleh *pre-warmer* Rust H-15 menit; TTL durasi ujian + 2 jam
Redis Cluster	`cbt:session:{exam_id}:{student_id}` (Jawaban aktif)	Di-flush periodik ke PG tiap 15-30 detik dan saat submit
Redis Cluster	`pos:idem:{idempotency_key}` (Guard tap ganda)	TTL 24 jam sebagai lapisan cepat sebelum constraint DB
Redis Cluster	`stream:events` & `bull:*`	Redis Streams (`MAXLEN ~100000`) + Consumer Group + DLQ
MinIO / S3	`ppdb-docs` , `erapor-signed` , `attendance-selfies`	Bucket **Private**, akses eksklusif lewat *Presigned URL* berumur 5–15 menit
MinIO / S3	`lms-media` , `library-ebooks`	Bucket **Private** dengan dukungan HTTP Range request untuk *streaming* video

BAB 5: Alur Proses Bisnis Kritis Lintas Layanan (Hybrid Workflows)

Visualisasi berikut merangkum bagaimana ketiga mesin bekerja sama secara otomatis dalam operasional sekolah sehari-hari:



Infografis 4 Alur Operasional Harian Smart School

5.1 Alur E-PPDB hingga One-Click Migration ke Buku Induk

1. Calon siswa mendaftar via Portal E-PPDB Web (Node.js) dan mengunggah berkas ke MinIO via *presigned URL*.
2. Node.js menerbitkan `invoices` biaya formulir dan menerima webhook pelunasan VA/QRIS.
3. Jika ada tes seleksi, peserta mengikuti *Placement Test* yang ditangani oleh **CBT Engine (Rust)** untuk ketahanan trafik serentak.
4. Setelah dinyatakan diterima, admin klik **One-Click Migration** di SIAKAD (Node.js). Dalam satu transaksi terkoordinasi, Node.js memutakhirkan status di `iam_master.students` (`ACTIVE`), menetapkan `nis` , mengaktifkan akses LMS, lalu memanggil endpoint internal Rust untuk menginisialisasi baris dompet di `high_speed_ops.wallets` .

5.2 Alur Penyusunan Jadwal Pelajaran Otomatis (`Node.js ↔ Python → Rust`)

1. Waka Kurikulum mengatur beban mengajar guru, ketersediaan lab/ruangan, dan preferensi waktu di Web SIAKAD (Node.js), lalu menekan **"Susun Jadwal Otomatis"**.
2. Node.js memanggil `POST /internal/ai/schedules/generate` pada layanan Python (FastAPI) dan langsung mengembalikan status `PROCESSING` ke browser pengguna.
3. Layanan Python menjalankan **Google OR-Tools CP-SAT Solver** untuk menghasilkan matriks jadwal yang **100% bebas bentrok** guru maupun ruangan.
4. Python menulis jadwal ke tabel `academic_lms.schedules` dan mengirim event `ScheduleGenerated` melalui Redis Streams.
5. Node.js memublikasikan jadwal ke Web/Mobile Guru & Siswa, sementara Rust menarik perubahan jadwal untuk memperbarui waktu bunyi **Automated IP Bell**.

5.3 Alur Presensi Gerbang Pagi & Early Warning System (`IoT → Rust → Node.js → Python`)

1. Pukul 06.30–07.00 WIB, ribuan siswa melakukan tap kartu RFID di gerbang sekolah secara bersamaan.
2. Perangkat ESP32 mengirim `{rfid_uid, device_id, timestamp}` via MQTT (QoS 1) ke broker EMQX yang dikonsumsi oleh **Service B (Rust)**.
3. Rust memvalidasi `rfid_uid` dari Redis/PostgreSQL dalam `< 3 ms`, menentukan status (`PRESENT` atau `LATE`), menulis ke `high_speed_ops.attendances`, mengirim ACK ke LED/Buzzer gerbang, dan mempublikasikan event `AttendanceRecorded` ke Redis Streams.
4. **Service A (Node.js)** membaca Redis Stream di latar belakang dan mengirimkan Push Notification (FCM) ke **Parent Super-App** dalam hitungan detik.
5. Pukul **08.00 WIB**, **Service C (Python)** menjalankan job harian EWS yang memindai tabel `attendances` dan `grades`. Jika terdeteksi pola alfa beruntun atau kemerosotan nilai tajam, Python menulis peringatan ke `finance_analytics.ews_student_risk_scores` yang langsung menyala pada Dashboard Kepala Sekolah & Guru BK.

5.4 Alur Ujian Serentak CBT Anti-Down (`Node.js → Rust`)

1. Guru menyusun bank soal dan jadwal sesi ujian di Web LMS (Node.js) yang tersimpan di `cbt_exams` dan `cbt_questions`.
2. **15 menit sebelum ujian (H-15m)**, layanan Rust melakukan *pre-warming*: mengambil seluruh butir soal, mengenkripsinya, dan memuatnya ke memori Redis.
3. Pukul 07.30 WIB, saat 1.000+ siswa menekan **"Mulai Ujian"** serentak, API Gateway mengarahkan seluruh trafik `/api/v1/engine/cbt/*` ke Rust.
4. Rust mengacak nomor soal per siswa, memvalidasi token `exam_browser_key`, serta menerima jawaban & *heartbeat* via WebSocket/Fast HTTP ke Redis dengan *batch write* berkala ke PostgreSQL—menjaga CPU/RAM `< 20%`.
5. Saat siswa klik **"Selesai"**, Rust melakukan *auto-grading* dalam hitungan mikrodetik dan menulis `auto_graded_score` ke `cbt_student_submissions` agar langsung tampil di SIAKAD (Node.js).

5.5 Alur Transaksi Kantin Cashless Anti-Race Condition (`Mobile → Node.js → POS → Rust`)

1. Orang tua melakukan *top-up* Rp100.000 via QRIS di Parent App dan menetapkan batas jajan harian anak (mis. Rp25.000/hari). Node.js memproses webhook pembayaran, lalu memanggil endpoint internal Rust untuk menambah saldo secara atomik.
2. Pada jam istirahat, siswa membeli makan siang Rp15.000 dan menempelkan kartu RFID di mesin POS Kantin.
3. Mesin POS mengirim permintaan ke `POST /api/v1/engine/pos/pay` (Service B Rust) dengan `idempotency_key` unik per keranjang transaksi.
4. Rust membuka transaksi PostgreSQL dengan `SELECT ... FOR UPDATE` pada baris `wallets`, memvalidasi kecukupan saldo, memeriksa total belanja hari ini `<= Rp25.000`, memotong saldo siswa, menambah saldo merchant kantin, dan mencatat `wallet_transactions` dalam `< 5 ms`. Tap ganda pada detik yang sama otomatis ditolak oleh `idempotency_key`.
5. Rust mempublikasikan event `WalletDeducted` ke Redis Streams, dan Node.js meneruskan notifikasi rincian jajanan ke ponsel orang tua.

BAB 6: Arsitektur Aplikasi Klien (Web, Mobile Flutter, & IoT)

6.1 Aplikasi Berbasis Web (`React.js + Vite + TypeScript`)

- **Tech Stack**: React.js (Vite), TypeScript, Tailwind CSS, Zustand / Redux Toolkit (state manajemen sesi & filter kompleks), TanStack Table (tabel data besar SIAKAD/Keuangan dengan paginasi server-side), dan WebSocket client (untuk CBT & Dashboard *real-time*).
- **Pembagian Portal**:
- **Portal E-PPDB Publik**: Formulir pendaftaran multi-langkah, unggah dokumen, status seleksi.
- **Portal SIAKAD & Manajemen (Admin/TU/Waka/Kepsek)**: Buku Induk, Generator Jadwal, E-Rapor TTE, E-Finance & BOS, E-BK, E-UKS, E-Library, Sarpras, serta Executive Dashboard + EWS.
- **Portal LMS & CBT (Guru & Siswa)**: Manajemen materi/tugas, penyusunan soal, dan antarmuka ujian CBT dengan validasi header/agent *Exam Browser Lock*.

6.2 Aplikasi Mobile Super-Apps (`Flutter / Dart — Android & iOS`)

Menggunakan arsitektur *Multi-Flavor Monorepo* dengan shared package (`core_auth`, `core_network`, `core_ui`, `core_qr`) untuk menjaga konsistensi autentikasi RS256 dan pemeliharaan kode:

1. **Parent Super-App**: Notifikasi presensi instan, pengajuan izin/sakit (unggah surat dokter ke MinIO), pantau nilai/PR, pembayaran tagihan SPP (VA/QRIS), *top-up* dompet anak & konfigurasi batas jajan harian, serta buku penghubung digital.
2. **Student App**: Kartu Pelajar Digital (*Dynamic QR* berkala untuk cegah pemalsuan), jadwal pelajaran harian, pengingat tenggat tugas LMS, pemesanan sesi konseling BK, dan status pinjaman perpustakaan.
3. **Teacher App**: Presensi pegawai menggunakan **GPS Geotagging (Mapbox API)** + swafoto (validasi radius koordinat sekolah), pengisian jurnal mengajar & absensi kelas `< 1 menit`, serta persetujuan izin siswa.

6.3 Integrasi Perangkat Keras & Sistem IoT

- **Smart Attendance Terminal (ESP32 / RPi + RFID/NFC + Biometrik)**:
- Terhubung via MQTT ke EMQX; memiliki *ring buffer* di memori *flash* internal untuk menyimpan ribuan log tap kartu saat Wi-Fi/internet terputus, dan melakukan *batch upload* otomatis ke endpoint Rust saat jaringan pulih.
- **Smart Cashless POS Kantin & Koperasi**:
- Perangkat tablet/terminal kasir dengan pembaca RFID/QRIS yang selalu membangkitkan UUID `idempotency_key` untuk setiap transaksi guna mencegah pemotongan ganda.
- **Self-Service Kiosk**: Anjungan peminjaman/pengembalian buku perpustakaan mandiri dan cetak surat keterangan aktif siswa (terhubung ke API Node.js).
- **Digital Signage, Automated IP Bell & Smartboard**: Layar pengumuman lobi dan bel terjadwal (sinkronisasi jadwal dari Rust/Python) serta papan tulis interaktif kelas terintegrasi SSO akun LMS guru.

BAB 7: Keamanan, Infrastruktur, Deployment, & Observability

7.1 Standar Keamanan Berlapis

1. **Isolasi Jaringan Internal (*Private Docker/K8s Network*)**: Hanya API Gateway (Nginx/Kong) dan Broker MQTT (EMQX) yang mengekspos port ke jaringan luar/publik. Service A, B, C, PostgreSQL, Redis, dan MinIO berjalan di *private subnet* tanpa akses langsung dari internet.
2. **Keamanan Kunci Asimetris (RS256) & Rotasi Kunci**: Hanya Service A (modul IAM/SSO) yang menyimpan `private.pem` (dalam Kubernetes Secret / Vault). Service B dan C hanya menerima `public.pem` (*read-only*). Mendukung mekanisme `kid` (*Key ID*) pada header JWT untuk rotasi kunci tanpa *downtime*.
3. **Keamanan Data Sensitif (E-BK & E-UKS)**: Catatan konseling psikologis siswa (`counseling_records`) dan rekam medis sensitif (`uks_visits`) dienkripsi *field-level* (AES-256-GCM) sebelum disimpan ke PostgreSQL dan hanya dapat didekripsi oleh peran berwenang (Guru BK / Petugas UKS).
4. **Integritas Ujian (Exam Browser Lock)**: Layanan Rust memverifikasi token perangkat/user-agent khusus Exam Browser pada setiap *handshake* WebSocket dan *heartbeat*, serta mencatat *event* pindah tab/kehilangan fokus ke `cheat_violation_count`.

7.2 Strategi Deployment (Cloud & Hybrid On-Premise Edge)

Seluruh layanan dikemas menggunakan **Docker** (multi-stage build):

- **Deployment Cloud (Docker Compose / Kubernetes)**: Ideal untuk operasional multi-sekolah (SaaS) atau sekolah dengan koneksi fiber stabil, dilengkapi *Horizontal Pod Autoscaler* (HPA) terpisah untuk Node.js, Rust, dan Python.
- **Hybrid On-Premise Edge (Local Mini-Server)**: Karena binari kompilasi Rust hanya berukuran **~20 MB** tanpa runtime berat, modul **CBT Engine** dan **IoT Ingestion** dapat dijalankan di mini-server lokal sekolah (Intel NUC / Mini PC) saat ujian akbar atau jaringan internet terganggu, lalu disinkronkan ke klaster utama begitu koneksi stabil.

7.3 Observability, Monitoring, & Strategi Pengujian

- **Metrics & Alerting**: **Prometheus + Grafana** memantau metrik kritis: latensi P99 Rust POS/RFID (`< 5 ms`), jumlah koneksi WebSocket CBT aktif, *PostgreSQL lock wait/deadlock*, *Redis memory & stream consumer lag*, serta status *online/offline* perangkat ESP32 di EMQX.
- **Distributed Tracing & Logging**: **OpenTelemetry** menyisipkan `X-Request-ID` / `traceparent` mulai dari Nginx/Kong menembus Node.js → gRPC/Redis → Rust/Python, dikumpulkan dalam **Grafana Loki** dengan format JSON terstruktur.
- **Load & Chaos Testing**: Menggunakan **k6** dan **MQTT-bench** untuk menyimulasikan:
- *Spike Test*: 1.000–2.000 siswa memulai ujian CBT & mengirim jawaban bersamaan setiap 5 detik.
- *Burst Test*: 1.500 pesan tap kartu RFID dalam jendela 10 menit melalui EMQX.
- *Concurrency Race Test*: 50 permintaan POS paralel menggunakan `idempotency_key` sama & berbeda pada satu dompet untuk membuktikan `0%` saldo minus/ganda.

BAB 8: Development Plan (Rencana Pengembangan 12 Bulan / 24 Sprint)

8.1 Rekomendasi Struktur Tim Pengembang

Peran	Jumlah	Tanggung Jawab Utama
Tech Lead / System Architect	1	Menjaga kepatuhan arsitektur polyglot, kontrak API/Event, review DB schema & keamanan
Backend Engineer — Node.js (NestJS)	2	Membangun Service A (IAM/SSO, E-PPDB, SIAKAD, LMS, E-Finance, E-BK/UKS/Library, Notif)
Backend Engineer — Rust (Axum)	1–2	Membangun Service B (CBT Engine, MQTT IoT Ingestion, Atomic POS Wallet, IP Bell sync)
AI / Optimization Engineer (Python)	1	Membangun Service C (OR-Tools Auto-Scheduling, Pandas/Scikit-learn EWS, Analitik)
Frontend Web Engineer (React/TS)	2	Membangun Portal E-PPDB, SIAKAD, LMS/CBT Web, E-Finance, dan Executive Dashboard
Mobile Engineer (Flutter)	2	Membangun Shared Core SDK & 3 aplikasi Flutter (Parent Super-App, Student, Teacher)
IoT / Embedded Engineer	1	Firmware ESP32/RPi untuk Terminal RFID Gerbang, POS Kantin, Kiosk, & IP Bell
DevOps / SRE	1	Docker/K8s, Kong/Nginx, EMQX, PostgreSQL/Redis/MinIO HA, CI/CD, & Observability
QA Engineer (Automation & Load)	1	E2E testing, API contract test, k6 load testing CBT/POS, security/RBAC verification

8.2 Strategi Repository & CI/CD

- **Struktur Monorepo Terorganisir** (memudahkan sinkronisasi kontrak antar bahasa):
- `apps/web-portal` (React + Vite)
- `apps/mobile-flutter` (Multi-flavor Flutter)
- `services/core-erp` (Node.js / NestJS)
- `services/high-speed-engine` (Rust / Axum)
- `services/ai-scheduling` (Python / FastAPI)
- `firmware/iot-terminals` (ESP32 / RPi)
- `infra/` (Migrations SQL terpusat, Docker Compose, Helm/K8s manifests, Kong config)
- `contracts/` (OpenAPI specs, gRPC `.proto`, JSON Schema Redis Streams)
- **CI/CD Pipeline**:
- Perubahan di `infra/migrations/` otomatis memicu test migrasi terhadap PostgreSQL ephemeral + verifikasi `sqlx` (Rust), Prisma/TypeORM (Node.js), dan SQLAlchemy (Python).
- Setiap PR wajib lulus *linting*, *unit test*, dan *build container* sebelum diizinkan *merge* ke `main`.

8.3 Peta Jalan 4 Fase (12 Bulan / 24 Sprint @ 2 Minggu)

Fase 1: Fondasi Core ERP & Tagihan (Bulan 1 – 3 | Sprint 1 – 6)

- **Fokus Teknologi**: Infrastruktur dasar, PostgreSQL SSOT, Node.js (NestJS), React.js (Vite), MinIO, Redis.
- **Sprint 1–2 (Setup & IAM)**: Setup Docker/Kong/PG/Redis/MinIO, skema migrasi awal `iam_master`, implementasi SSO JWT RS256 & RBAC di Node.js, shell Web Portal React.
- **Sprint 3–4 (E-PPDB & Buku Induk)**: Modul formulir E-PPDB, upload dokumen MinIO, *One-Click Migration* ke `students`, master Tahun Ajaran, Rombel, & Mapel (`academic_lms`).
- **Sprint 5–6 (SIAKAD Dasar & E-Finance)**: Pengelolaan nilai dasar, penerbitan `invoices` SPP, integrasi webhook Payment Gateway (VA/QRIS), laporan tagihan & kas dasar, UAT Fase 1.
- **Exit Criteria Fase 1**: Siswa dapat mendaftar via E-PPDB, dimigrasi 1-klik ke Buku Induk SIAKAD, login via SSO ke portal web, dan orang tua/siswa dapat melunasi tagihan SPP otomatis via VA/QRIS.

Fase 2: Mesin IoT Kehadiran, Penjadwalan Pintar, & Mobile Apps (Bulan 4 – 6 | Sprint 7 – 12)

- **Fokus Teknologi**: Rust (Axum), EMQX MQTT, ESP32 RFID, Python (FastAPI + OR-Tools), Flutter (Dart + Mapbox).
- **Sprint 7–8 (IoT Ingestion & Auto-Scheduling Core)**: Setup EMQX & skeleton Rust Service B (verifikasi `public.pem` RS256 + MQTT worker), firmware ESP32 RFID + offline buffer, serta model OR-Tools CP-SAT di Python Service C.
- **Sprint 9–10 (Integrasi Jadwal & Presensi Real-Time)**: Alur lengkap `Node.js ↔ Python` untuk generate jadwal bebas bentrok; alur `ESP32 → EMQX → Rust → Redis → Node.js FCM`; inisiasi Flutter Shared Core.
- **Sprint 11–12 (Rilis Mobile Apps V1)**: Rilis Parent App (notif hadir & izin digital), Teacher App (presensi GPS Mapbox + swafoto + jurnal mengajar `< 1 menit`), Student App (Kartu Pelajar Dynamic QR & jadwal), uji coba lapangan di 1 gerbang sekolah.
- **Exit Criteria Fase 2**: Tap kartu RFID di gerbang masuk `< 3 ms` memicu notif di ponsel orang tua, jadwal 1 sekolah tersusun tanpa bentrok oleh Python, dan 3 aplikasi Flutter berjalan di Android/iOS.

Fase 3: Pembelajaran Digital & Mesin Ujian CBT Berkinerja Tinggi (Bulan 7 – 9 | Sprint 13 – 18)

- **Fokus Teknologi**: LMS (Node.js), CBT Engine (Rust + Redis), E-Rapor TTE (Node.js), IP Bell & Smartboard.
- **Sprint 13–14 (LMS & Bank Soal)**: Modul LMS di Node.js (materi, streaming video MinIO, tugas, forum, bank soal `cbt_exams` / `cbt_questions`), tampilan LMS di Web & Student App.
- **Sprint 15–16 (CBT Engine Rust)**: Implementasi *pre-warmer* soal terenkripsi ke Redis, pengacakan soal per siswa, WebSocket/HTTP sync jawaban, validasi *Exam Browser Lock*, *auto-grading* mikrodetik, dan sinkronisasi hasil ke SIAKAD.
- **Sprint 17–18 (E-Rapor TTE, IP Bell, & Load Test CBT)**: Generator PDF E-Rapor + integrasi TTE, sinkronisasi jadwal IP Bell di Rust + Signage/Smartboard SSO, serta **Load Test k6 1.000+ peserta CBT serentak** (Cloud & Edge Mini-Server).
- **Exit Criteria Fase 3**: Simulasi ujian CBT 1.000+ siswa serentak berjalan lancar dengan CPU/RAM `< 20%`, E-Rapor terbit dengan TTE valid, dan bel sekolah otomatis sinkron dengan jadwal OR-Tools.

Fase 4: Ekosistem Kantin Cashless, Layanan Kesiswaan, & Kecerdasan EWS (Bulan 10 – 12 | Sprint 19 – 24)

- **Fokus Teknologi**: Atomic Wallet & POS (Rust), E-BK/E-UKS/E-Library/Sarpras (Node.js), EWS & Analitik (Python), Kios Mandiri.
- **Sprint 19–20 (Atomic Wallet & POS Kantin)**: Implementasi `wallets` (`BIGINT`), *top-up* QRIS via Node.js → Rust, batas jajan harian di Parent App, endpoint POS Rust (`SELECT ... FOR UPDATE` + `idempotency_key`), serta aplikasi kasir POS Kantin & notifikasi rincian jajan.

- **Sprint 21–22 (Layanan Kesiswaan, Perpus & Sarpras)**: Modul E-BK (enkripsi AES-256), E-UKS, Alumni Tracer Study, E-Library OPAC & Kios Mandiri, serta inventaris QR & tiket kerusakan Sarpras.
- **Sprint 23–24 (Early Warning System & Final Hardening)**: Implementasi model EWS harian (08.00 WIB) di Python, Executive Dashboard Kepala Sekolah/Yayasan, *penetration testing*, *chaos/race condition testing*, dan serah terima produksi penuh.
- **Exit Criteria Fase 4**: Transaksi kantin `< 5 ms` bebas *double-deduct* dan saldo minus, seluruh modul kesiswaan/sarpras aktif, serta Dashboard Eksekutif menampilkan skor risiko EWS secara akurat setiap pagi.

8.4 Manajemen Risiko & Mitigasi Teknis

Risiko	Dampak	Strategi Mitigasi
Konflik Skema DB antar 3 Bahasa	Query gagal saat runtime (terutama `sqlx` Rust)	Terapkan **Single Migration Authority** (`infra/migrations/`), wajib *schema check* di CI untuk Node.js, Rust, & Python.
Putus Koneksi Internet Sekolah saat Jam Masuk / Ujian	Presensi gerbang gagal atau CBT terhenti	Firmware ESP32 menyimpan log ke *flash* lokal + *batch retry*; binari Rust CBT (`~20 MB`) dideploy di **Local Mini-Server** sekolah.
Race Condition / Tap Ganda di POS Kantin	Saldo siswa terpotong dua kali atau menjadi minus	Gunakan `BIGINT` rupiah, `SELECT ... FOR UPDATE`, validasi batas harian di dalam transaksi DB, dan wajib `idempotency_key` unik.
Ketidakakuratan Data Awal EWS (Cold Start)	Skor EWS belum akurat di bulan-bulan awal	Mulai dengan aturan *rule-based scoring* berbasis bobot alfa & penurunan nilai pada Fase 4, lalu latih model ML Scikit-learn setelah data 1–2 semester terkumpul.

BAB 9: Task List & Work Breakdown Structure (WBS)

Kode Workstream: `INF` (Infra/DevOps), `DB` (Database SSOT), `CORE` (Service A Node.js), `ENG` (Service B Rust), `AI` (Service C Python), `WEB` (Frontend React), `MOB` (Mobile Flutter), `IOT` (Hardware/Firmware), `QA` (Testing & Security).

9.1 Task List Fase 1: Fondasi Core ERP & Tagihan (Bulan 1 – 3)

ID	Tugas (Task)	Stream	Prioritas	Est. (MD)	Dependensi	Kriteria Penerimaan (Acceptance Criteria)
`INF-01`	Setup Monorepo, Docker Compose lokal, & CI/CD pipeline	`INF`	`P0`	4	-	Repo terstruktur; pipeline lint, test, & Docker build berjalan otomatis di setiap PR.
`INF-02`	Deploy & konfigurasi Kong/Nginx API Gateway (SSL, Rate Limit, Routing)	`INF`	`P0`	4	`INF-01`	Rute `/api/v1/core/*`, `/engine/*`, `/ai/*` terpisah dengan TLS & rate limit aktif.
`INF-03`	Provisioning PostgreSQL 16+, Redis Cluster, & MinIO S3 (Private Network)	`INF`	`P0`	5	`INF-01`	DB, Redis, & MinIO hanya dapat diakses dari internal network; backup otomatis terjadwal.
`DB-01`	Setup Single Migration Tool & DDL awal skema `iam_master`, `academic_lms`, `finance_analytics`	`DB`	`P0`	5	`INF-03`	Seluruh tabel inti Fase 1 (users, RBAC, students, classrooms, invoices) termigrasi bersih.
`CORE-01`	Implementasi Modul IAM/SSO (JWT RS256 `private.pem`, Refresh Token, RBAC Guards)	`CORE`	`P0`	6	`DB-01`	Login menghasilkan JWT RS256 valid; endpoint memblokir akses lintas peran (RBAC).
`CORE-02`	Integrasi MinIO *Presigned URL* & BullMQ Notification Dispatcher (Email/WA/FCM)	`CORE`	`P1`	4	`INF-03`, `CORE-01`	Upload/download berkas privat aman via presigned URL; antrean notif asinkron berjalan.
`CORE-03`	Implementasi Modul E-PPDB & *One-Click Migration* ke Buku Induk SIAKAD	`CORE`	`P0`	8	`CORE-01`, `CORE-02`	Data pendaftar yang diterima otomatis menjadi baris `students` aktif & akun siswa terbuat.
`CORE-04`	Implementasi Modul SIAKAD Dasar (Buku Induk, Tahun Ajaran, Rombel, Mapel, Dapodik Export)	`CORE`	`P0`	8	`CORE-03`	Admin dapat mengelola kelas/rombel, mutasi siswa, dan ekspor format siap Dapodik/EMIS.

ID	Tugas (Task)	Stream	Prioritas	Est. (MD)	Dependensi	Kriteria Penerimaan (Acceptance Criteria)
`CORE-05`	Implementasi Modul E-Finance (Generator Tagihan SPP, Webhook Payment Gateway VA/QRIS, Kas/BOS)	`CORE`	`P0`	8	`CORE-01`, `DB-01`	Pembayaran VA/QRIS tervalidasi HMAC webhook dan otomatis mengubah status `invoices` → `PAID`.
`WEB-01`	Setup React + Vite + TS Shell, Auth SSO, & Manajemen Pengguna/RBAC	`WEB`	`P0`	5	`CORE-01`	Pengguna login sekali dan mendapat menu dinamis sesuai role.
`WEB-02`	Bangun UI Portal E-PPDB & Modul Admin SIAKAD Dasar	`WEB`	`P0`	10	`WEB-01`, `CORE-03`, `CORE-04`	Alur pendaftaran siswa baru hingga penempatan rombel berjalan E2E di browser.
`WEB-03`	Bangun UI E-Finance (Manajemen Tagihan SPP, Pembayaran, & Laporan BOS)	`WEB`	`P0`	7	`WEB-01`, `CORE-05`	Bendahara dapat membuat tagihan massal & memantau rekonsiliasi pembayaran *real-time*.
`QA-01`	E2E Test, RBAC Security Check, & Payment Webhook Idempotency Test Fase 1	`QA`	`P0`	5	`WEB-02`, `WEB-03`	0 temuan *broken access control*; webhook duplikat tidak menggandakan pencatatan bayar.

9.2 Task List Fase 2: Mesin IoT Kehadiran, Penjadwalan Pintar, & Mobile Apps (Bulan 4 – 6)

ID	Tugas (Task)	Stream	Prioritas	Est. (MD)	Dependensi	Kriteria Penerimaan (Acceptance Criteria)
`INF-04`	Deploy Broker MQTT EMQX + TLS/Auth & Redis Streams Consumer Groups	`INF`	`P0`	4	`INF-03`	EMQX menerima koneksi perangkat uji dengan autentikasi device ID & ACL topik.
`DB-02`	Migrasi DDL `high_speed_ops.attendances` & indeks `identity_cards.rfid_uid`	`DB`	`P0`	3	`DB-01`	Indeks komposit `(date, user_id)` & unique `rfid_uid` terpasang untuk lookup sub-ms.
`ENG-01`	Bootstrap Service B (Rust/Axum), SQLx Pool, & Verifikasi JWT RS256 (`public.pem`)	`ENG`	`P0`	5	`INF-02`, `CORE-01`	Service Rust memvalidasi token terbitan Node.js secara lokal tanpa query ke PostgreSQL.
`ENG-02`	Implementasi Worker Ingesti MQTT Absensi RFID + Endpoint Batch Offline Sync	`ENG`	`P0`	7	`ENG-01`, `INF-04`, `DB-02`	Validasi tap kartu `< 3 ms`, insert `attendances`, kirim ACK MQTT & publish `AttendanceRecorded`.

ID	Tugas (Task)	Stream	Prioritas	Est. (MD)	Dependensi	Kriteria Penerimaan (Acceptance Criteria)
`CORE-06`	Consumer Event `AttendanceRecorded` (FCM Push ke Ortu) & API Izin/Jurnal Guru	`CORE`	`P0`	6	`ENG-02`, `CORE-02`	Notifikasi hadir/telat masuk ke ponsel orang tua `< 5 detik` setelah tap kartu di gerbang.
`AI-01`	Bootstrap Service C (Python/FastAPI) & Verifikasi JWT RS256 (`public.pem`)	`AI`	`P0`	3	`INF-02`, `CORE-01`	Endpoint `/api/v1/ai/*` dapat diakses dengan token SSO dan terhubung ke PG & Redis.
`AI-02`	Implementasi Engine Auto-Scheduling (Google OR-Tools CP-SAT Solver)	`AI`	`P0`	10	`AI-01`, `CORE-04`	Menghasilkan jadwal ke `schedules` dengan **0 bentrok** guru/kelas/lab & kirim event `ScheduleGenerated`.
`CORE-07`	Integrasi Trigger & Review Jadwal Otomatis di SIAKAD (`Node.js ↔ Python`)	`CORE`	`P1`	4	`AI-02`	Waka Kurikulum dapat menekan "Susun Jadwal Otomatis" dan memantau progres hingga aktif.
`WEB-04`	Bangun UI Manajemen Jadwal Pintar & Monitoring Kehadiran Live di Web SIAKAD	`WEB`	`P1`	6	`CORE-06`, `CORE-07`	Admin melihat matriks jadwal tanpa bentrok serta rekap presensi harian siswa & guru.
`IOT-01`	Pengembangan Firmware ESP32 Terminal RFID (MQTT + Flash Offline Ring Buffer)	`IOT`	`P0`	8	`INF-04`, `ENG-02`	Perangkat membaca RFID, menyalakan LED/buzzer, dan menyimpan antrean lokal saat Wi-Fi mati.
`MOB-01`	Setup Monorepo Flutter (3 Flavors), Shared Auth RS256, FCM, & Dynamic QR SDK	`MOB`	`P0`	6	`CORE-01`	Fondasi 3 aplikasi mobile siap dengan SSO login, refresh token, dan generator QR dinamis.
`MOB-02`	Bangun Fitur V1 Parent App (Notif Hadir, Izin Digital, Pantau Nilai/SPP)	`MOB`	`P0`	8	`MOB-01`, `CORE-06`	Orang tua menerima notif tap kartu *real-time* & dapat mengajukan izin sakit/izin digital.
`MOB-03`	Bangun Fitur V1 Teacher App (Presensi GPS Mapbox + Swafoto, Jurnal `< 1 mnt`) & Student App	`MOB`	`P0`	9	`MOB-01`, `CORE-06`	Guru absen dalam radius GPS sekolah + swafoto ke MinIO; siswa melihat Kartu Digital & jadwal.
`QA-02`	Uji Burst MQTT 1.500 Tap Kartu, Uji Offline Sync ESP32, & Verifikasi Solver Jadwal	`QA`	`P0`	5	`IOT-01`, `AI-02`, `MOB-02`	0 data tap hilang saat simulasi putus internet; 0 bentrok pada hasil jadwal OR-Tools.

9.3 Task List Fase 3: Pembelajaran Digital & Mesin Ujian CBT Berkinerja Tinggi (Bulan 7 – 9)

ID	Tugas (Task)	Stream	Prioritas	Est. (MD)	Dependensi	Kriteria Penerimaan (Acceptance Criteria)
`DB-03`	Migrasi DDL `cbt_exams`, `cbt_questions`, `cbt_student_submissions`, & `e_rapor_documents`	`DB`	`P0`	4	`DB-01`	Skema CBT & E-Rapor TTE siap dengan constraint unik per `(exam_id, student_id)`.
`CORE-08`	Implementasi Modul LMS (Materi, Video Streaming MinIO, Tugas, Forum, Bank Soal CBT)	`CORE`	`P0`	10	`DB-03`, `CORE-02`	Guru dapat mengunggah materi/video, mengelola tugas, dan menyusun bank soal untuk CBT.
`ENG-03`	Implementasi CBT Pre-Warmer (H-15m Enkripsi Redis) & Pengacakan Soal per Siswa	`ENG`	`P0`	6	`ENG-01`, `DB-03`, `CORE-08`	Paket soal ujian masuk ke Redis terenkripsi sebelum jam mulai; urutan soal unik tiap siswa.
`ENG-04`	Implementasi CBT Live Engine (WebSocket/Fast HTTP Sync, Exam Lock Heartbeat, Auto-Grader)	`ENG`	`P0`	9	`ENG-03`	Jawaban tersimpan ke Redis + *batch flush* PG; auto-grading menghitung skor saat submit.
`CORE-09`	Consumer `CbtExamSubmitted`, Modul Penilaian Akhir & Generator PDF E-Rapor ber-TTE	`CORE`	`P0`	8	`ENG-04`, `CORE-04`	Nilai CBT otomatis masuk rekap SIAKAD; PDF E-Rapor ditandatangani TTE & tersimpan di MinIO.
`ENG-05`	Implementasi Sinkronisasi Jadwal Automated IP Bell (`schedules` → MQTT/HTTP Bel)	`ENG`	`P1`	4	`ENG-01`, `AI-02`	Perubahan jadwal di Python otomatis memperbarui jadwal bunyi bel jaringan sekolah.
`WEB-05`	Bangun UI Portal LMS (Guru/Siswa), Antarmuka Ujian CBT (Exam Lock), & Cetak E-Rapor	`WEB`	`P0`	10	`CORE-08`, `ENG-04`, `CORE-09`	Siswa mengerjakan ujian CBT di browser terkunci; guru mengelola nilai dan E-Rapor TTE.
`MOB-04`	Update Student App (Materi & Pengingat Tugas LMS) & Parent App (Unduh E-Rapor)	`MOB`	`P1`	6	`CORE-08`, `CORE-09`	Siswa mendapat pengingat tugas LMS; orang tua dapat melihat/mengunduh PDF E-Rapor.
`IOT-02`	Integrasi Perangkat IP Bell, Digital Signage Lobi, & Smartboard Kelas (SSO LMS)	`IOT`	`P2`	6	`ENG-05`, `CORE-08`	Bel berbunyi tepat waktu sesuai jadwal; guru login ke Smartboard langsung mengakses materi LMS.

ID	Tugas (Task)	Stream	Prioritas	Est. (MD)	Dependensi	Kriteria Penerimaan (Acceptance Criteria)
INF-05	Paketkan Deployment Edge Mini-Server untuk Binari Rust CBT (~20 MB) & Sinkronisasi	INF	P1	4	ENG-04	Node CBT Rust dapat dideploy di mini-server lokal sekolah dan melakukan sinkronisasi hasil.
QA-03	Load Test k6 1.000+ Peserta CBT Serentak & Uji Keamanan Exam Browser Lock	QA	P0	6	ENG-04, WEB-05	CPU & RAM server < 20% saat 1.000 siswa aktif serentak; pelanggaran fokus tercatat akurat.

9.4 Task List Fase 4: Ekosistem Kantin Cashless, Layanan Kesiswaan, & Kecerdasan EWS (Bulan 10 – 12)

ID	Tugas (Task)	Stream	Prioritas	Est. (MD)	Dependensi	Kriteria Penerimaan (Acceptance Criteria)
DB-04	Migrasi DDL wallets, wallet_transactions, ews_student_risk_scores, & tabel BK/UKS/Perpus/Asset	DB	P0	4	DB-01	Kolom saldo bertipe BIGINT, idempotency_key Unique, dan tabel kesiswaan termigrasi.
ENG-06	Implementasi Core Wallet & POS Kantin (SELECT ... FOR UPDATE , Batas Harian, Idempotency)	ENG	P0	8	ENG-01, DB-04	Pemotongan saldo < 5 ms ; 0% saldo minus; tap ganda pada detik sama ditolak otomatis.
CORE-10	Top-up Saldo QRIS → Rust Credit, Config Daily Limit Ortu, & Notif Rincian Jajan (WalletDeducted)	CORE	P0	6	ENG-06, CORE-05	Orang tua isi saldo & atur limit harian; notifikasi item jajanan terkirim setelah transaksi POS.
CORE-11	Implementasi Modul E-BK (Enkripsi AES-256), E-UKS, & Alumni Tracer Study	CORE	P1	8	DB-04, CORE-01	Data konseling BK terenkripsi di DB; rekam medis/alergi UKS & survei alumni berfungsi penuh.
CORE-12	Implementasi Modul E-Library (OPAC, Sirkulasi, E-Book) & Asset/Sarpras (QR + Ticketing)	CORE	P1	8	DB-04, CORE-02	Katalog OPAC, peminjaman buku, scan QR aset, dan tiket perbaikan sarpras berjalan E2E.
AI-03	Implementasi Engine Early Warning System (Job 08.00 WIB Pandas/Scikit-learn) & Rekomendasi LMS	AI	P0	9	AI-01, DB-04, CORE-09	Skor attendance_risk_score & academic_drop_score terhitung harian dan publish EwsAlertRaised .
AI-04	Implementasi Endpoint Agregasi Analitik Berat Dashboard Kepala Sekolah & Yayasan	AI	P1	5	AI-03	API menyajikan tren akademik, kedisiplinan, dan arus kas sekolah secara agregat < 500 ms .

ID	Tugas (Task)	Stream	Prioritas	Est. (MD)	Dependensi	Kriteria Penerimaan (Acceptance Criteria)
`WEB-06`	Bangun UI Aplikasi Kasir POS Kantin, Layanan E-BK/UKS/Perpus/Sarpras, & Kios Mandiri UI	`WEB`	`P0`	10	`ENG-06`, `CORE-11`, `CORE-12`	Kasir kantin memproses bayar RFID/QR; siswa dapat meminjam buku & cetak surat di Kios.
`WEB-07`	Bangun UI Executive Dashboard Kepala Sekolah & Yayasan dengan Panel Alert EWS	`WEB`	`P0`	7	`AI-03`, `AI-04`	Grafik performa sekolah & daftar siswa berisiko (`CRITICAL` / `MEDIUM`) tampil interaktif.
`MOB-05`	Update Parent App (Top-up Dompet, Limit Jajan, Riwayat Kantin) & Student App (Booking BK, Perpus)	`MOB`	`P0`	7	`CORE-10`, `CORE-11`, `CORE-12`	Orang tua mengontrol jajan anak dari ponsel; siswa dapat memesan konseling & cek pinjaman buku.
`IOT-03`	Integrasi Perangkat Keras POS Kantin (RFID/QRIS Reader) & Kios Mandiri (Scanner + Thermal Printer)	`IOT`	`P1`	6	`ENG-06`, `WEB-06`	Perangkat POS & Kios berfungsi stabil di lingkungan operasional kantin dan perpustakaan.
`QA-04`	Concurrency/Race-Condition Test POS Wallet, Penetration Test Penuh, & Final UAT Produksi	`QA`	`P0`	7	Seluruh modul Fase 4	Lulus uji 50 request paralel POS tanpa saldo minus; 0 temuan kerentanan kritikal/tinggi.

Checklist Kesiapan Go-Live Ekosistem

- ☐ Seluruh skema database terpusat (`iam_master`, `academic_lms`, `high_speed_ops`, `finance_analytics`) tervalidasi tanpa migrasi konflik.
- ☐ Kunci `private.pem` RS256 tersimpan aman di Vault/Secret khusus Service A; Service B & C hanya memakai `public.pem`.
- ☐ Private Network Docker/K8s terverifikasi: DB, Redis, MinIO, dan 3 backend tidak dapat diakses langsung dari internet publik.
- ☐ Uji beban CBT `1.000+` siswa serentak dan uji *burst* RFID `06.30–07.00` WIB memenuhi target SLA (`< 20%` CPU/RAM dan `< 3 ms` ingesti).
- ☐ Uji transaksi POS Kantin mengonfirmasi `0%` *race condition*, presisi rupiah `BIGINT`, dan pemblokiran *double-tap* via `idempotency_key`.
- ☐ Backup otomatis PostgreSQL & MinIO serta *runbook* pemulihan bencana (*Disaster Recovery*) telah diuji.